Anatomy of a Smart Pointer

Michael VanLoon

CPPcon 2014

michaelv@NonComposMentis.net

What is a smart pointer?

- It’s a container for a pointer
  - It holds your pointer and gives it back to you
  - It facilitates RAII
    - Constructs with reasonable defaults
    - Cleans up pointer at appropriate time
- (Ideally) properly conveys interface semantics
  - Shared, unique, etc.

What is a smart pointer?

- It’s a proxy for a pointer
  - It stands in for the pointer in many cases where you might use the raw pointer
  - In many cases it can act as if it is the original pointer type

Various kinds of smart pointers

- “Generic” smart pointers:
  - auto_ptr
  - scoped_ptr
  - unique_ptr
  - shared_ptr
  - weak_ptr
  - _com_ptr_t
  - CComPtr

Various kinds of smart pointers

- Domain-specific smart pointers:
  - std::string
  - _bstr_t
  - _variant_t

The different kinds of reference counting

- Object-based reference counting
  - The count is stored in the object
  - Microsoft COM model
  - Sometimes referred to as “intrusive” (boost::intrusive_ptr)
- Container-based reference counting
  - The count is stored in the smart pointer container
  - shared_ptr style

Object-based reference counting

- Advantages
  - Very simple smart pointer logic
  - Can easily mix with raw pointer usage
  - Object has greater visibility into its lifetime and cleanup
  - Lighter weight overall
- Disadvantages
  - Outside the COM world, less familiar
  - Outside the COM world, no ready-made solution
  - Object itself becomes slightly bigger and more complex

Container-based reference counting

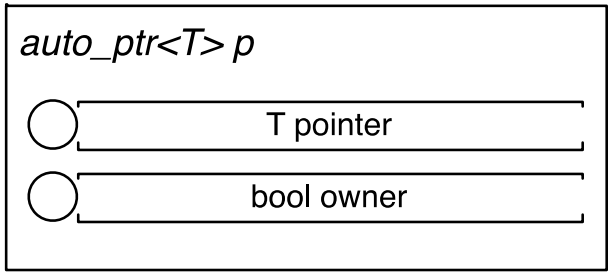
- Advantages
  - Can contain pointer to any objet; no added object knowledge or complexity
  - Very standard and widely used
- Disadvantages
  - Difficult to mix with raw pointer usage
  - Heavier weight overall because of complexity of good shared_ptr implementation

How it works: object-based reference counting

- Sorry this talk is too short, so here's what you get:
  - Stick an atomic count in an object
  - Provide methods to add and release references
  - Object deletes itself when reference goes to zero

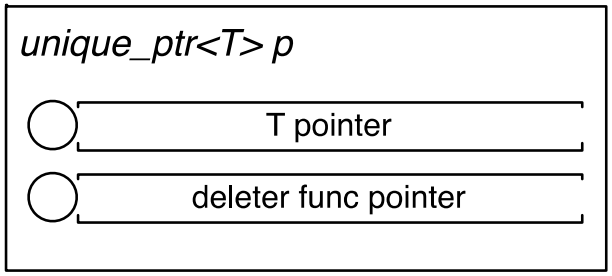
How it works: auto_ptr

- Contains raw pointer and an ownership flag
- Ownership is transferred to last auto_ptr to contain pointer
- Problematic and error-prone ownership semantics, so it's deprecated!



How it works: scoped_ptr and unique_ptr

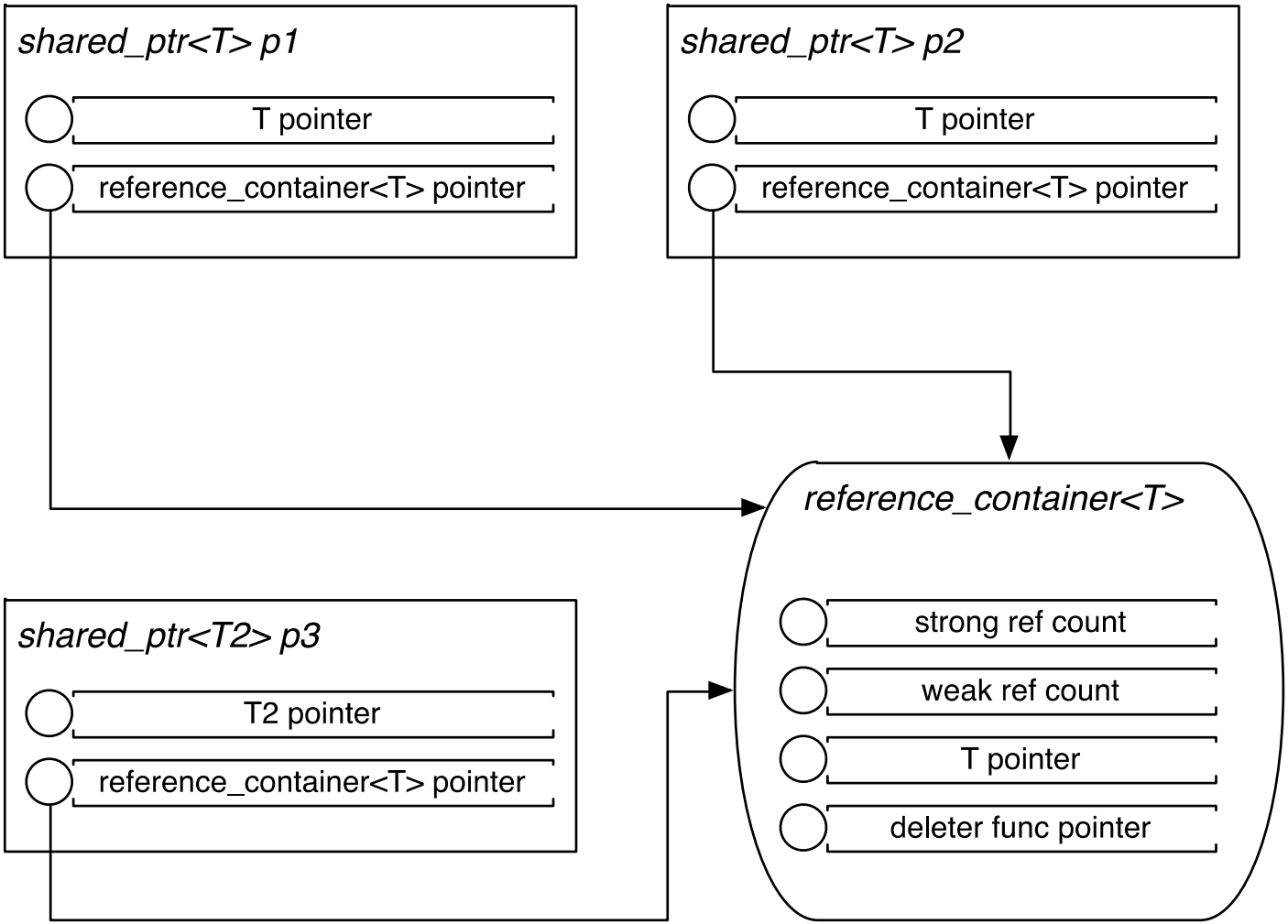
- Contains raw pointer
- Does now allow transfer of ownership (except via move in unique_ptr), which simplifies logic
- unique_ptr optionally contains pointer to custom deleter func, which will be called instead of delete if desired



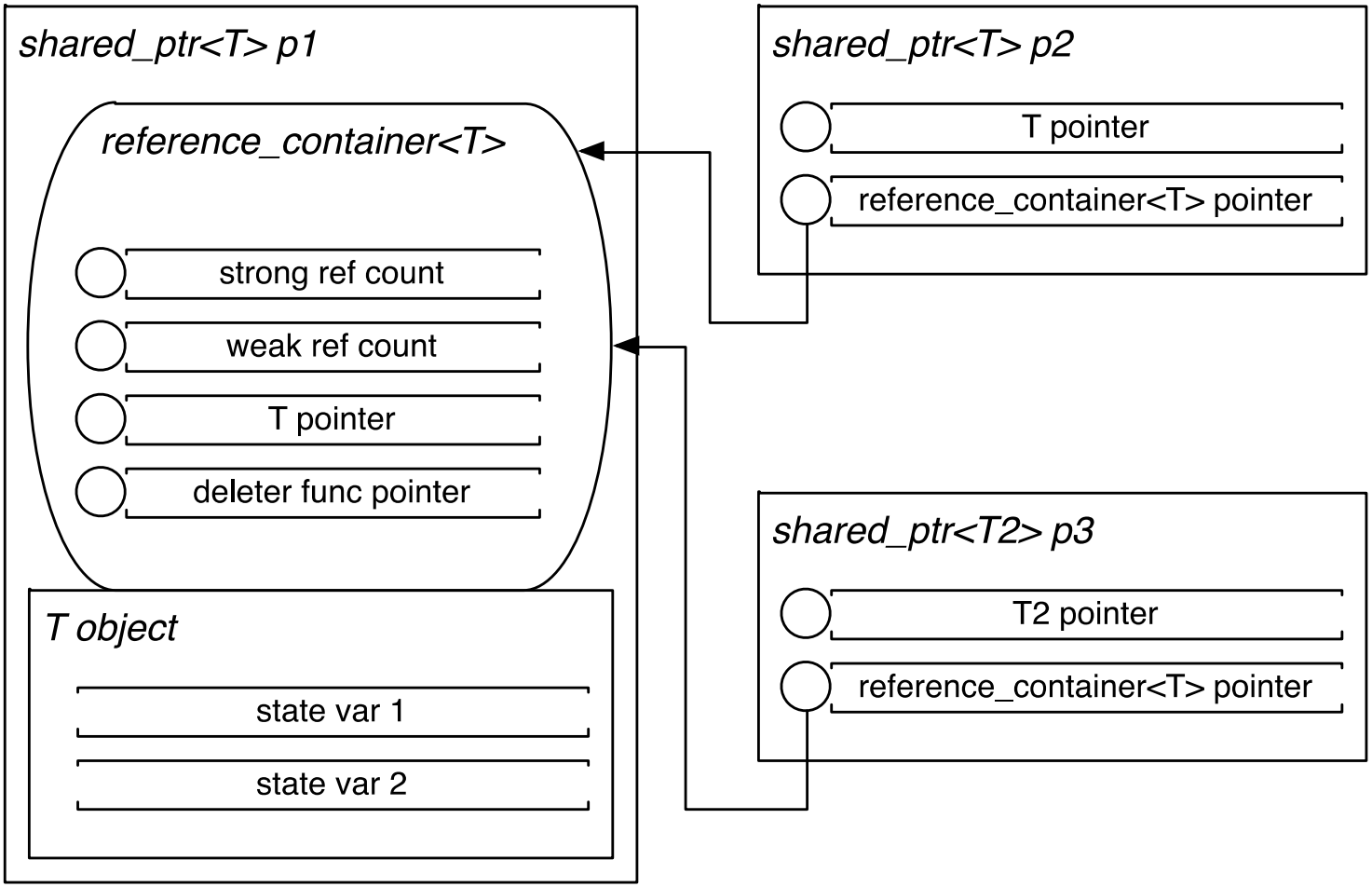
How it works: shared_ptr

- High level overview:
- Contains a pointer and a reference count that is shared among all instances
- Contains optional custom deleter
- Last releaser decrements reference to zero, and object is deleted

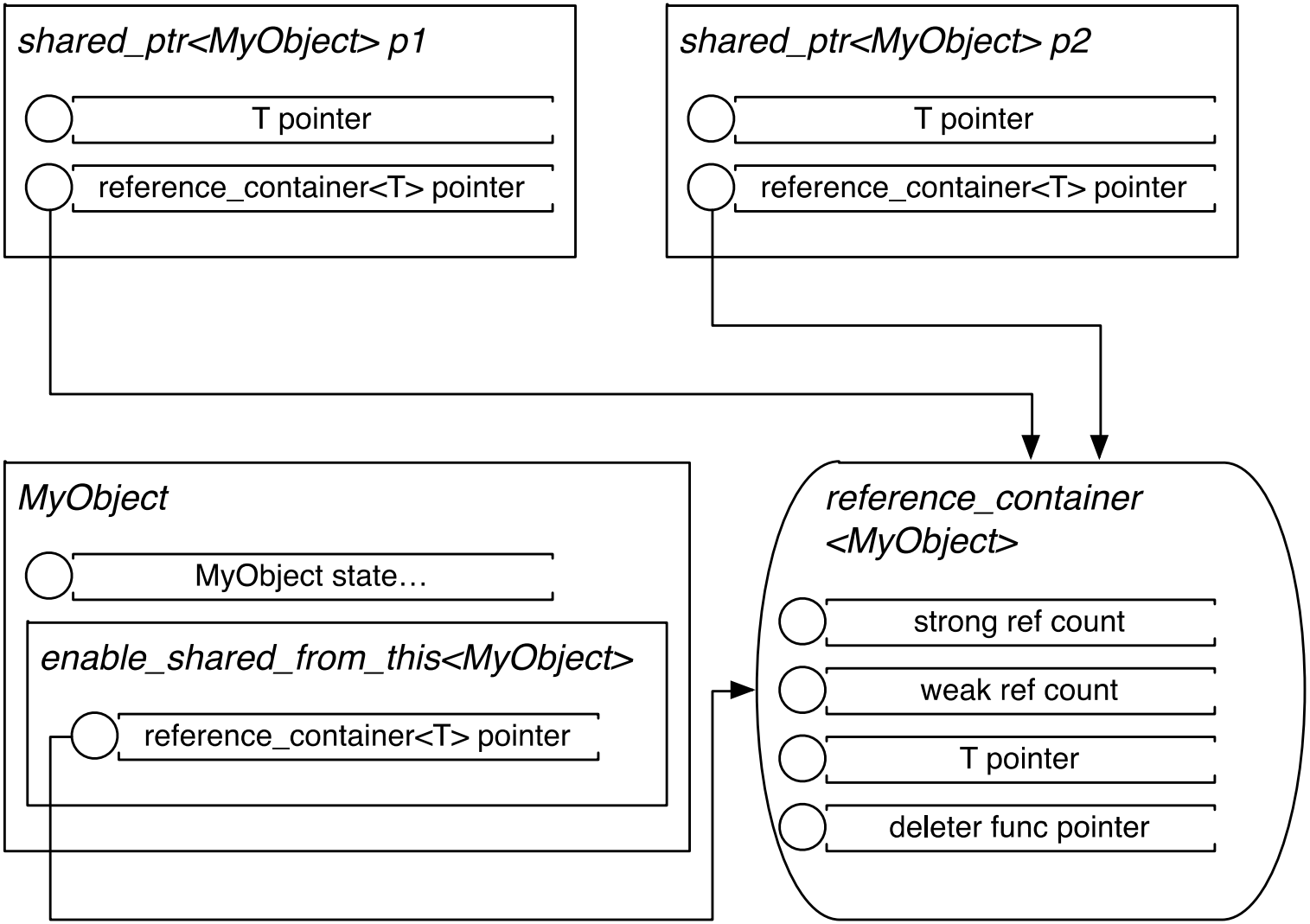
Example shared_ptr implementation



Example make_shared optimization



Example enable_shared_from_this



Summary

- Use a smart pointer. It avoids the explicit, problematic manual delete
- Prefer unique_ptr (or boost::scoped_ptr if you don't have C++11 yet) because of its efficiency and simplicity if possible
- Use shared_ptr for any variables that need shared semantics
- Use make_shared when possible; it may be more efficient, and avoids explicit new
- Consider object-based reference counting for cases where it fits a paradigm better